



南京林业大学

《林木生物信息学》

上机报告

题 目： 系统发育树中目标类群单系性的自动判定

学 院： 信息科学技术学院、人工智能学院

专 业： 计算机科学与技术

班 级： 23508014

学 号： 2351610105

姓 名： 方泽宇

指导教师： 王则夫

二〇二六 年 五 月 一 日

目录

一、项目背景与目的 - 2 -

二、基本概念与定义 - 4 -

三、输入输出设计 - 7 -

四、算法与实现思路 - 11 -

五、测试与结果展示 - 20 -

六、讨论与扩展思考 - 28 -

七、总结 - 30 -

八、进一步思考的扩展问题 - 31 -

一、项目背景与目的

1.系统发育树在分类学研究中的重要性

系统发育树（phylogenetic tree）是描述生物类群之间进化关系的重要工具。它以树状结构的形式，直观展示了不同样本、物种或基因序列之间的亲缘关系和分化顺序。在林木生物信息学及相关分类学研究中，系统发育树被广泛应用于以下方面：

- 1) 物种鉴定与分类：通过构建系统发育树，可以判断未知样本与已知类群的亲缘关系，辅助物种界定。
 - 2) 进化历史推断：树形结构反映了类群之间的分化和演化路径，有助于理解物种或种群的起源与扩散过程。
 - 3) 群体遗传分析：在林木遗传育种中，系统发育树可用于分析种质资源的亲缘关系、指导杂交亲本选择和种质保护策略。
 - 4) 基因家族进化研究：通过基因树与物种树的比较，可以识别基因复制、丢失等进化事件。
- 因此，准确理解和分析系统发育树的结构，是开展许多生物信息学研究和进化生物学研究的前提。

2. 单系群在系统发育分析中的意义

在系统发育学中，“单系群”（monophyletic group）是一个核心概念。一个类群被称为单系群，当且仅当它包含一个共同祖先及其所有后代。在树结构中，这对应于存在某一个内部节

点，该节点下的全部叶节点恰好构成该组样本，且不包含任何其他样本。单系群的重要性体现在：

- 1) 分类学的自然性：现代分类学强调只有单系群才能被正式命名为分类单元（如属、科等）。并系群（paraphyletic group）或多系群（polyphyletic group）被认为不能真实反映进化关系。
- 2) 进化推断的可靠性：在比较基因组学、性状演化分析等研究中，只有基于单系群的比较才能避免因样本混杂而导致的假阳性或偏误。
- 3) 群体归属验证：在林木遗传资源研究中，若某一育种群体（如 A1 - A10）在多棵树上均能形成单系群，则表明该群体具有稳定的遗传一致性，反之则可能暗示存在基因交流、分类错误或样本混淆。

3. 人工判断单系群的局限性

在传统的工作流程中，研究人员通常需要逐棵查看系统发育树的图形（如使用 FigTree、MEGA 等软件），通过视觉判断某组样本是否聚为单系。然而，这种方法存在明显局限：

- 1) 效率低下：当树的数量较多时（例如 bootstrap 重复抽样生成的数千棵树），人工逐棵检查几乎不可行。
- 2) 主观性强：不同研究者对“是否构成单系”的判断可能存在差异，尤其是边界情况（如个别样本位置模糊）。
- 3) 难以批量统计：如果要获得“单系群比例”这一重要统计量，人工方式极易出错且无法复现。

无法自动识别异常样本：人工方法很难系统性地统计哪些样本最常“脱离”目标单系群。

4. 项目目标：自动化判定单系群

为解决上述问题，本项目旨在开发一个自动化程序，能够：

- 1) 读取以 Newick 格式存储的系统发育树文件（单棵或多棵）；
- 2) 根据给定的目标样本集合（例如 A1 - A10），自动判断每棵树中该集合是否构成严格单系群；
- 3) 进一步实现宽松判定规则（例如允许最多 k 个目标样本缺失，但不允许混入非目标样本），以适应实际研究中存在的少量异常样本；
- 4) 在多棵树输入的情况下，统计严格通过比例与宽松通过比例，并自动汇总最常导致判定失败的异常样本。

5) 通过该程序，研究者可以将大量树文件的单系群判定工作从“人工看图”转变为“自动批量计算”，显著提高分析效率、结果的可重复性和统计可靠性。这也是本项目在林木生物信息学课程中实际应用价值的重要体现。

二、基本概念与定义

为了准确理解本程序对单系群的判定逻辑，有必要先明确系统发育树中的几个核心概念及其在本项目中的具体含义。

1. 系统发育树 (Phylogenetic Tree)

系统发育树是一种用于表示样本、类群或基因序列之间进化关系的树状结构。在本项目中，可以将其理解为一种层级化的数据结构：

- 1) 树的末端对应输入的具体样本（如 A1、A2、B7、C）；
- 2) 树的分支连接关系反映了样本在拓扑结构中的相对亲缘远近；
- 3) 树的根为整棵树提供一个时间或进化方向的参考点。

本项目处理的树文件采用 Newick 格式，该格式使用括号表示层级、逗号分隔并列节点、分号表示树结束。例如，树 ((A1,A2),(B1,B2),C); 表示 A1 与 A2 先聚为一支，B1 与 B2 先聚为一支，这两支再与 C 并列。

注：本项目基础版本忽略分支长度信息（如 A1:0.02），解析器在读取时自动剥离长度值和注释，只关注树的拓扑结构。

2. 叶节点与内部节点

- 1) 叶节点 (Leaf / Tip)：位于树最末端的节点，对应一个具体的样本名称。例如，在 Newick 字符串中出现的所有样本名（A1、A2、B1、C 等）都是叶节点。叶节点没有子节点。
- 2) 内部节点 (Internal Node)：分支交汇处的节点，代表一个假想的共同祖先。每个内部节点对应一个由其所有后代叶节点组成的子树。在 Newick 格式中，每一对匹配的括号 (...) 都隐含地定义了一个内部节点。
- 3) 区分这两类节点是判断单系群的基础，因为单系群的判定核心就是寻找是否存在一个内部节点，其下的全部叶节点恰好为目标集合。

3. 子树 (Subtree)

子树是指从某个内部节点出发，向下包含其全部后代叶节点的一部分树结构。每个内部节点唯一地定义了一棵子树，而该子树的“叶节点集合”就是该节点下所有末端样本的集合。

例如，在树 ((A1,A2,A3),(B1,B2),C) 中：

- 1) 左侧内部节点对应的子树叶节点集合为 {A1, A2, A3}；
- 2) 右侧内部节点对应的子树叶节点集合为 {B1, B2}；
- 3) 根节点对应的子树叶节点集合为 {A1, A2, A3, B1, B2, C}。
- 4) 判断单系群时，本质上就是检查：是否存在某个内部节点，它对应的叶节点集合与目标集合恰好相等（严格判定），或在给定宽容度下近似相等（宽松判定）。

4. 外类群及其作用

外类群 (Outgroup) 是指在系统发育分析中已知与研究对象亲缘关系较远、用于为树“定根”的一个或一组样本。在本项目中，样本 C 被指定为外类群。

外类群在本项目中的具体作用包括：

- 1) 辅助理解树的方向：包含外类群的树，其根通常位于外类群与其余样本之间的分支上，这有助于区分哪些样本“更早分化出来”。
- 2) 不影响单系群判定的核心逻辑：本程序在判断 A 组 (A1 - A10) 是否构成单系群时，不需要对外类群做特殊处理——外类群 C 仅仅被视为一个普通的“非目标样本”。如果某个内部节点下同时包含 A 组样本和 C，则该节点一定不满足严格单系群的条件（因为混入了非 A 样本）。
- 3) 辅助宽松判定的参照：在某些情况下，外类群的存在可以帮助判断 A 组是否“整体上相对外类群形成独立分支”，但这属于进阶分析，本项目的基础判定规则中不做强制要求。
- 4) 简单来说，外类群 C 在本项目中可以被理解为一个不属于目标组 A 的普通样本，它在判定逻辑中与 B 组样本没有本质区别——只要混入，就会破坏单系性。

5. 单系群的程序化定义

在本项目中，单系群不再是一个模糊的生物学概念，而是一个可计算、可严格判断的逻辑条件。根据判定标准的严格程度，我们区分以下两种定义：

1) 严格判定 (Strict Criterion)

定义：存在某个内部节点，其下游全部叶节点构成的集合与目标样本集合完全相同。

等价条件：

- a. 该内部节点下包含且仅包含全部目标样本；
- b. 不包含任何非目标样本（包括外类群 C 和 B 组样本）；
- c. 所有目标样本必须全部位于该节点下，不能有任何一个目标样本在该节点之外。

典型不通过原因：

- a. 不存在叶节点集合与目标集合完全一致的内部节点（目标样本缺失）；
- b. 包含全部目标样本的节点下混入了非目标样本；
- c. 目标样本被分散在多个分支中，不存在能包含全部目标样本的单一内部节点。

2) 宽松判定 (Tolerant Criterion)

在实际研究中，由于测序误差、基因不完全谱系分选 (ILS)、水平基因转移或建树方法本身的局限性，偶尔会出现少量目标样本“偏离”主体分支的情况。为了区分“整体稳定”与“整体不单系”这两种情况，我们引入宽松判定规则。

定义（本项目采用的规则）：存在某个内部节点，其下游叶节点集合满足以下两个条件：

- a. 包含目标集合中的至少 $|\text{target_set}| - k$ 个样本（即允许最多 k 个目标样本缺失）；
- b. 不包含任何非目标样本（即不允许混入 B 组或 C 等样本）。

* 其中 k 为可设置的宽容度参数，通常取 1 或 2。

说明：

- a. 宽松判定只允许缺失少量目标样本，但不允许混入非目标样本。这一选择是为了保证“该分支仍然是一个纯净的目标子群”，只是规模略小于完整的目标集合。
- b. 如果某节点下混入了非目标样本，即使包含了几乎全部目标样本，也不满足宽松判定。
- c. 典型通过场景：

A1 - A10 中，A9 和 A10 因长枝吸引等原因被“甩”到了树的其他位置，但其余 8 个 A 组样本仍然在某个内部节点下形成一个完整且纯净的分支（不含任何 B/C），且该分支内未混入其他样本。

3) 两种判定的对比总结

判定类型	包含样本数	混入非目标样本	适用场景
严格判定	全部 (10/10)	不允许	标准版本，要求完美单系
宽松判定	$\geq \text{目标数} - k$	不允许	容忍少量异常样本，关注整体结构

注意：本项目的宽松规则与一些文献中允许“少量混入”的定义不同，我们选择了“只允许

缺失、不允许混入”的规则。这样做的理由是：在实际问题中，混入非目标样本往往意味着该分支在分类学上不干净，而缺失少数目标样本可能仅仅是数据或方法造成的偶然现象。程序中已将该规则明确写入注释，保证透明可复现。

三、输入输出设计

1. 输入格式说明

本程序的输入包括三个部分：Newick 树文件、目标样本列表和宽容度参数 k 。下面分别进行说明。

1) Newick 树文件（可支持多棵树）

程序接受一个或多个以 Newick 格式存储的系统发育树文件。Newick 格式使用括号表示层级关系，逗号分隔并列节点，分号表示树的结束。本程序能够自动识别并处理以下两种情况：

a. 单棵树文件：文件中仅包含一行有效的 Newick 字符串。

b. 多棵树文件：文件中每行包含一棵树，或使用分号分隔的多棵树。

在实际测试中，程序处理的输入为 100 个独立的树文件，命名方式为 `tree_001.nwk` 至 `tree_100.nwk`，每个文件包含一棵系统发育树。这些树文件存放于指定目录下，程序通过遍历目录自动读取。同时，本程序也支持单文件多棵树模式：用户可通过命令行参数 `--tree`（或 `-t`）指定一个包含多棵以分号（`;`）分隔的 Newick 树的文本文件，程序会自动识别路径类型（文件或目录）并采用相应的加载策略。

文件示例（`tree_001.nwk`）：

```
((A1,A2,A3,A4,A5,A6,A7,A8,A9),B1,B2,C);
```

或更复杂的嵌套结构：

```
((A1,A2),(A3,A4)),((A5,A6,A7),(A8,A9,A10)),(B1,B2),C);
```

注：本程序在解析 Newick 格式时会自动剥离分支长度信息（如 `A1:0.02` → `A1`），同时支持去除 NHX 注释（如 `A1[&color=red]` → `A1`）和引号包裹的名称，只关注树的拓扑结构，因为单系群判定仅依赖于样本之间的相对位置关系，与分支长度无关。

2) 目标样本列表

目标样本列表用于指定需要判断是否构成单系群的样本集合。在本项目中，目标样本为 `A1` 至 `A10`，共 10 个样本。

a. 输入方式：

输入方式：程序支持通过命令行参数 `--target`（或 `-g`）传入，多个样本名以空格分隔；若不指定则使用 `config.py` 中的默认值。例如：`python main.py --target A1 A2 A3 A4 A5`

例如：`target_group = {"A1", "A2", "A3", "A4", "A5", "A6", "A7", "A8", "A9", "A10"}`

支持的样本命名规则：

本程序假设样本名称不包含括号、逗号、分号等 Newick 格式中的保留字符。

样本名称区分大小写，A1 与 a1 被视为不同样本。

3) 宽容度参数 k

宽松判定中允许缺失的目标样本最大数量，记为 k。在本项目中，k 的取值为 1 或 2，推荐默认值为 2。

含义：

当 $k = 1$ 时，允许最多 1 个 A 组样本未包含在某个纯净的内部节点中，但该节点下不得混入任何非目标样本。

当 $k = 2$ 时，允许最多 2 个 A 组样本缺失。

程序实现方式：

程序实现方式：k 通过命令行参数 `--k`（或 `-k`）传入，默认值为 2。例如：`python main.py --k 1k` 作为函数参数传入判定逻辑中，例如：`check_tolerant_monophyly(root, target_set, k=2)`

2. 输出内容说明

本程序的输出分为三个层次：每棵树的详细判定结果、不通过原因说明、以及多棵树输入时的整体统计汇总。

1) 每棵树的判定结果

对于每一棵输入的树，程序输出以下信息：

strict: 严格判定结果，取值为 `True` 或 `False`。

tolerant: 宽松判定结果，取值为 `True` 或 `False`。

missing samples: 当严格判定不通过时，输出导致判定失败的具体缺失样本（仅在缺失样本时显示；若因混入非目标样本导致失败，则显示空集）。

2) 实际输出示例（来自程序运行结果）：

trees/tree_001.nwk

strict:	False
tolerant:	True
missing samples:	{A1}

解读：在 tree_001 中，严格单系群不成立（因为 A1 缺失），但宽松判定成立（因为只缺失了 1 个样本，且未混入非目标样本）。

trees/tree_006.nwk

strict:	True
tolerant:	True
missing samples:	set()

解读：在 tree_006 中，严格单系群成立，宽松判定自然也成立，缺失样本为空集。

trees/tree_005.nwk

strict:	False
tolerant:	False
missing samples:	set()

解读：在 tree_005 中，严格和宽松判定均不通过。missing samples 为空集，说明失败原因并非“缺失样本”，而是在包含大量 A 组样本的节点下混入了非目标样本（如 B1、B2 或 C），或者 A 组样本被分散在多个不连通的分支中。

2) 不通过原因说明

当严格判定不通过时，程序根据失败的具体情形输出不同的原因。根据项目要求，主要区分以下几种情况：

失败类型 输出原因说明 对应的 missing samples 表现

混入非目标样本 “包含全部目标样本的节点中混入了非目标样本” missing samples: set()

目标样本分散 “目标样本被分散在多个分支中，不存在能包含全部目标样本的单一内部节点” missing samples: set()

不存在完全一致节点 “不存在叶节点集合与目标集合完全一致的内部节点（目标样本缺失：具体样本）” missing samples: {'A1', 'A5'}

注：在上述程序输出中，missing samples 为空集并不一定意味着严格判定通过，也可能是

因混入或分散导致的失败。程序中三种失败原因的输出与项目说明中的示例完全对应。

3) 多棵树情况下的统计结果

当输入为多棵树时（如本测试中的 100 棵树），程序在输出完所有树的逐棵结果后，会输出整体统计汇总。

实际输出示例（来自程序运行结果）：

树总数:	100
Strict 比例:	0.45
Tolerant 比例:	0.72

最容易破坏单系的样本:

A1	6
A5	6
A10	6
A3	4
A4	4
A9	3
A8	3
A6	3
A7	2
A2	2

统计指标说明：

a.树总数：实际成功解析并参与判定的树文件数量（本测试为 100 棵）。

b.Strict 比例：严格判定通过的比例。计算公式为 通过严格判定的树的数量 / 树总数。本测试结果为 0.45，表示在 100 棵树中，有 45 棵中 A1 - A10 构成了完美单系群。

c.Tolerant 比例：宽松判定通过的比例。计算公式为 通过宽松判定的树的数量 / 树总数。本测试结果为 0.72，表示在 100 棵树中，有 72 棵满足宽松单系条件（允许最多 k 个缺失，此处 k=2）。

4) 异常样本汇总:

- a.程序统计了在所有未通过严格判定的树中, 哪些样本最常“缺失”。
- b.输出格式为 样本名 缺失次数, 并按缺失次数降序排列。
- c.本测试结果显示: A1、A5、A10 均缺失 6 次, 并列成为最常破坏单系的样本; A3 和 A4 缺失 4 次, A9、A8、A6 缺失 3 次, A7 和 A2 缺失 2 次。
- d.该统计对研究者具有重要指导意义: 如果某个样本在多棵树中反复游离于主体分支之外, 可能提示该样本存在样本混淆、测序错误或真实的杂交渗入。

四、算法与实现思路

1 系统总体架构设计

1) 模块化分层架构

整个系统采用模块化分层设计, 将不同功能职责分离到独立模块中。主程序 `main.py` 作为入口层, 通过 `argparse` 解析命令行参数, 负责协调整个流程; `parser` 层负责 Newick 格式解析 (含分支长度剥离); `tree` 层定义核心数据结构; `analysis` 层实现单系性判定算法; `utils` 层提供文件操作工具 (支持文件/目录两种输入模式自动识别); `stats` 层负责结果统计与图表生成; `config` 层集中管理默认配置参数。这种设计使得各模块职责清晰, 便于维护和扩展。

```
config.py
1  """
2  config.py
3
4  项目参数配置
5  """
6
7  # A组样本
8  TARGET_GROUP = {
9      "A1", "A2", "A3", "A4", "A5",
10     "A6", "A7", "A8", "A9", "A10"
11 }
12
13 # 宽松允许缺失
14 TOLERANCE_K = 2
15
16 # 数据目录
17 TREE_DIR = "trees"
```

2) 数据流处理流程

程序从指定的路径读取树数据 (支持目录或单文件两种模式): 若路径为目录, 则遍历其中所有 `.nwk` 文件; 若路径为文件, 则按分号 (;) 拆分得到多棵树。通过 `file_utils` 的 `load_trees`

函数统一加载，每个 Newick 字符串经过 `parse_newick` 函数转换为 `TreeNode` 树结构，然后分别进行严格单系性和宽松单系性判定，最后通过 `ResultStats` 类汇总统计结果并输出报告。

3) 配置驱动设计

程序采用双层参数策略：默认参数在 `config.py` 中集中管理（如 `TARGET_GROUP`、`TOLERANCE_K`、`TREE_DIR`），同时支持通过命令行参数（`--target`、`--k`、`--tree`、`--output`）在运行时覆盖默认值。这种设计兼顾了开箱即用（无需任何参数即可运行）和灵活配置（可通过 CLI 调整参数）的需求。

2 核心数据结构设计

1) `TreeNode` 节点类

```
class TreeNode:

    def __init__(self, name=None):
        # 节点名称 (叶节点才有)
        self.name = name

        # 子节点列表
        self.children = []

    def is_leaf(self): # 判断是否为叶节点
        return len(self.children) == 0

    def add_child(self, node): # 添加子节点
        self.children.append(node)

    def get_all_leaves(self): # 获取该节点子树下所有叶节点名称
        if self.is_leaf():
            return {self.name}
        leaves = set()
        for child in self.children:
            leaves.update(child.get_all_leaves())
        return leaves
```

本程序使用 `TreeNode` 类表示系统发育树的节点结构，叶节点存储样本名称，内部节点通过 `children` 链接子节点

a 节点属性设计

每个 `TreeNode` 包含 `name` 和 `children` 两个核心属性。`name` 用于存储叶节点的样本名称，内部节点的 `name` 为 `None`；`children` 是一个列表，存储所有子节点引用。通过 `is_leaf` 方法判断节点类型，当 `children` 为空时为叶节点。

b 递归遍历方法

`get_all_leaves` 方法采用深度优先搜索策略递归收集子树下所有叶节点名称。对于叶节点直接返回包含自身名称的集合；对于内部节点则遍历所有子节点，合并各子树的叶节点集合。该方法使用 Python 的 `set` 数据结构，天然支持集合运算，便于后续的单系性判定。

c 树构建接口

`add_child` 方法提供向节点添加子节点的能力，在解析过程中动态构建树的层次结构。

2) 集合数据结构选择

使用 Python 内置的 `set` 类型存储叶节点集合和目标样本集合，主要基于以下考虑：集合的相等性判断可以直接比较两个集合是否完全相同；集合的 `issubset` 方法可以高效判断子集关系；集合的差集运算可以快速计算缺失样本；集合的哈希特性保证元素唯一性，避免重复样本问题。

3 Newick 格式解析算法

```
def parse_newick(newick_str):
    stack = []
    current_node = TreeNode()
    token = ""

    for ch in newick_str:
        if ch == "(":
            # 进入新层级→创建子节点→入栈
            new_node = TreeNode()
            current_node.add_child(new_node)
            stack.append(current_node)
            current_node = new_node
        elif ch == ",":
            # 兄弟节点→处理 token 创建叶节点
            if token.strip():
                leaf = TreeNode(_clean_name(token)) # 清理分支长度后创建叶节点
                current_node.add_child(leaf)
                token = ""
        elif ch == ")":
            # 结束层级→处理 token→出栈
            if token.strip():
                leaf = TreeNode(_clean_name(token)) # 清理分支长度后创建叶节点
                current_node.add_child(leaf)
                token = ""
            current_node = stack.pop()
        elif ch == ";":
            continue
        else:
```

```
token += ch # 累积节点名称
```

```
return current_node.children[0]
```

遇到 '(' 创建子节点并入栈，遇到 ')' 出栈回退父节点

1) 基于栈的迭代解析策略

采用单遍扫描配合栈结构的迭代算法解析 Newick 字符串，时间复杂度为 $O(n)$ ，其中 n 为字符串长度。算法通过 `_clean_name` 函数在创建节点时自动剥离分支长度（:数字）和 NHX 注释（[...]），确保节点名称纯净。

2) 字符处理规则

a 左括号处理

当遇到 '(' 字符时，创建新的内部节点，将其作为当前节点的子节点加入，然后将当前节点压入栈中保存，最后将当前节点指针移动到新创建的节点。这表示进入一个新的子树层级。

b 逗号处理

当遇到 ',' 字符时，如果 `token` 缓冲区中有累积的字符，则创建一个叶节点并添加到当前节点的子节点列表中，然后清空 `token` 缓冲区。逗号表示同一层级的兄弟节点分隔。

c 右括号处理

当遇到 ')' 字符时，首先检查并处理可能存在的叶节点名称（某些 Newick 格式允许在右括号前有标签），然后从栈中弹出父节点，将当前节点指针回退到父节点。这表示退出当前子树层级。

d 分号处理

分号 ';' 是 Newick 格式的结束标记，算法直接忽略该字符继续处理。

e 名称字符处理

对于其他字符，统一追加到 `token` 缓冲区中，这些字符构成节点名称的一部分。

3) 根节点提取

解析完成后，初始的空节点会有一个子节点，该子节点即为真正的树根。通过返回 `current_node.children[0]` 获取完整的树结构。

我们给出了 MRCA 辅助函数，用于找到包含目标样本的最小包含结点。

```
def find_mrca(root, target_set):
```

```
    """
```

```
    找到包含全部目标样本的"最小包含节点" (MRCA)。
```

```
    即：该节点包含全部目标样本，但其任意子节点都不包含全部目标样本。
```

```
    """
```

```

nodes = find_all_nodes(root)
for node in nodes:
    leaves = node.get_all_leaves()
    if not target_set.issubset(leaves):
        continue
    # 检查是否有子节点也能包含全部目标样本
    child_also_contains = False
    for child in node.children:
        if target_set.issubset(child.get_all_leaves()):
            child_also_contains = True
            break
    if not child_also_contains:
        return node, leaves
return None, None

```

4 严格单系性判定算法

```

def check_strict_monophyly(root, target_set):
    """
    严格单系群判定
    条件: 存在某节点, 其所有叶节点 == target_set
    """

```

失败时区分三种原因（与项目说明示例对应）：

- 1) 不存在叶节点集合与目标集合完全一致的内部节点（目标样本缺失）
- 2) 混入非目标样本：包含全部目标样本的节点中混入了非目标样本
- 3) 目标样本被分散：目标样本过于分散，无单一节点能包含全部

"""

```

nodes = find_all_nodes(root)

```

第一步：完美匹配检查

```

for node in nodes:
    if node.get_all_leaves() == target_set:
        return True, "存在一个节点, 其叶节点集合与目标集合完全一致"

```

第二步：检查"目标样本缺失"

寻找最大的纯净（只含目标样本）节点

```

MIN_PURE = len(target_set) - tolerance_k # 使用命令行参数传入的 k 值
best_pure_count = 0
best_pure_missing = None
for node in nodes:
    leaves = node.get_all_leaves()
    if leaves.issubset(target_set) and len(leaves) > best_pure_count:
        best_pure_count = len(leaves)
        best_pure_missing = target_set - leaves

```

```

    if best_pure_count >= MIN_PURE and best_pure_missing:
return False, (f'不存在叶节点集合与目标集合完全一致的内部节点 ("
f'目标样本缺失: {sorted(best_pure_missing)}}, "

# 第三步: 检查MRCA 是否混入非目标样本
mrca, mrca_leaves = find_mrca(root, target_set)
if mrca is not None:
    extra = mrca_leaves - target_set
    if extra:
        return False, f"包含全部目标样本的节点中混入了非目标样本:
{sorted(extra)}"

# 第四步: 目标样本被分散
return False, "目标样本被分散在多个分支中, 不存在能包含全部目标样本的单一内部节点"

```

1) 算法原理

严格单系群的定义要求存在树中的一个节点, 其子树下包含且仅包含目标集合中的所有样本, 不多不少。即该节点的所有叶节点集合必须与目标集合完全相等。

2) 全节点遍历策略

通过 `find_all_nodes` 函数对树进行深度优先遍历, 收集树中所有节点 (包括内部节点和叶节点) 到一个列表中。遍历过程使用递归实现, 对于每个节点先将其加入列表, 然后递归处理所有子节点。

3) 精确匹配判定

遍历所有节点, 对每个节点调用 `get_all_leaves` 获取其子树下的全部叶节点集合, 然后与目标集合进行相等性比较。一旦找到匹配的节点立即返回 `True` 和成功原因说明; 如果遍历完所有节点都未找到匹配, 则返回 `False` 和失败原因。

4) 时间复杂度分析

设树中节点总数为 N , 每个节点的叶节点收集操作最坏情况下需要 $O(L)$ 时间 (L 为叶节点总数), 因此总时间复杂度为 $O(N*L)$ 。在实际的系统发育树中, 由于树通常是平衡的, 平均性能会优于最坏情况。

5 宽松单系性判定算法

```

# === 宽松判定 (k=2) ===
def check_tolerant_monophyly(root, target_set, tolerance_k):

```

```

"""条件: 1. 节点下不能出现非目标样本 2. 允许缺失 <= k"""
nodes = find_all_nodes(root)
for node in nodes:
    leaves = node.get_all_leaves()
    if not leaves.issubset(target_set): # 不允许混入非目标
        continue
    missing = target_set - leaves
    if len(missing) <= tolerance_k: # 允许最多 k 个缺失
        return True, missing
return False, set()

```

1) 算法设计理念

宽松判定放宽了严格相等的要求，允许目标集合中的部分样本缺失，但不允许混入非目标样本。这种设计更符合实际生物信息学场景，因为测序数据可能存在样本丢失或命名不一致的情况。

2) 双重约束条件

a 纯度约束

节点下的所有叶节点必须是目标集合的子集，即不能出现任何非目标样本。通过 `leaves.issubset(target_set)` 进行判断，如果不满足则跳过该节点。这个约束保证了单系群的纯净性。

b 缺失容忍约束

计算目标集合与节点叶节点集合的差集，得到缺失的样本集合。如果缺失数量不超过预设的容差阈值 `tolerance_k`，则判定为通过。缺失数量的计算通过 `len(target_set - leaves)` 实现。

3) 最优节点选择

算法遍历所有节点，返回第一个满足条件的节点对应的缺失样本集合。如果需要找到缺失最少的节点，可以在遍历过程中记录最优解，但当前实现采用首次匹配策略以提高效率。

4) 返回值设计

成功时返回 `True` 和缺失样本集合，方便用户了解具体哪些样本缺失；失败时返回 `False` 和空集合，保持接口一致性。

6 统计分析模块

1) 增量式统计设计

`ResultStats` 类采用增量式统计方式，每处理一棵树就调用 `add_result` 方法更新统计数据。这

种方式避免了存储所有中间结果，节省内存空间，特别适合处理大规模数据集。

2) 统计指标计算

维护三个核心计数器：`total` 记录处理的树总数，`strict_pass` 记录通过严格判定的数量，`tolerant_pass` 记录通过宽松判定的数量。比例计算在输出时动态执行，使用除法运算符得到浮点数结果。

3) 异常样本追踪

使用 `defaultdict(int)` 类型的 `failure_samples` 字典记录每个样本在宽松判定中失败的次数。每次判定后遍历缺失样本集合，对每个样本的计数加 1。最终按失败次数降序排序输出，帮助识别最容易破坏单系性的样本。

4) 结果输出格式化

`print_summary` 方法按照固定格式输出统计报告，包括树总数、严格比例、宽松比例以及异常样本排行榜。使用 `sorted` 函数配合 `lambda` 表达式实现按值降序排序。

7 文件处理与批处理机制

1) 目录扫描策略

`load_tree_files` 函数使用 `os.listdir` 扫描指定目录，通过 `endswith` 方法过滤出 `.nwk` 后缀的文件，并使用 `sorted` 函数保证文件按字母顺序处理，确保结果的可重现性。

2) 文件读取封装

`read_file` 函数封装了标准的文件打开、读取、关闭流程，使用 `with` 语句确保文件资源正确释放，返回去除首尾空白后的字符串内容。

3) 批量处理循环

主程序使用 `for` 循环依次处理每个树文件，对每个文件执行完整的解析-判定-统计流程。这种串行处理方式简单可靠，易于调试和理解。

```

26     newick = read_file(path)
27
28     root = parse_newick(newick)
29
30     strict, reason = check_strict_monophyly(
31         root,
32         config.TARGET_GROUP
33     )
34
35     tolerant, missing = check_tolerant_monophyly(
36         root,
37         config.TARGET_GROUP,
38         config.TOLERANCE_K
39     )
40
41     print("\n", path)
42     print("strict:", strict)
43     print("tolerant:", tolerant)
44     print("missing samples:", missing)
45
46     stats.add_result(strict, tolerant, missing)
47
48     stats.print_summary()

```

8 算法优化与扩展性考虑

1) 集合运算优化

充分利用 Python 集合的高效运算特性，如相等比较、子集判断、差集计算等都是底层 C 实现，比手动遍历比较更高效。

2) 早期终止策略

在严格和宽松判定中，一旦找到满足条件的节点就立即返回，避免不必要的后续遍历，在最坏情况下仍能正确处理，在最佳情况下显著提升性能。

3) 可扩展性设计

模块化设计使得替换解析器、增加新的判定算法、修改统计指标都变得容易。例如可以增加基于分支长度的加权判定，或者增加多目标集合的同时判定功能。

4) 内存效率考虑

采用流式处理方式，每次只保留一棵树在内存中，处理完成后即可释放，适合处理大量树文件的场景。如果需要全局分析，可以考虑将所有树结构缓存，但这会增加内存消耗。

五、测试与结果展示

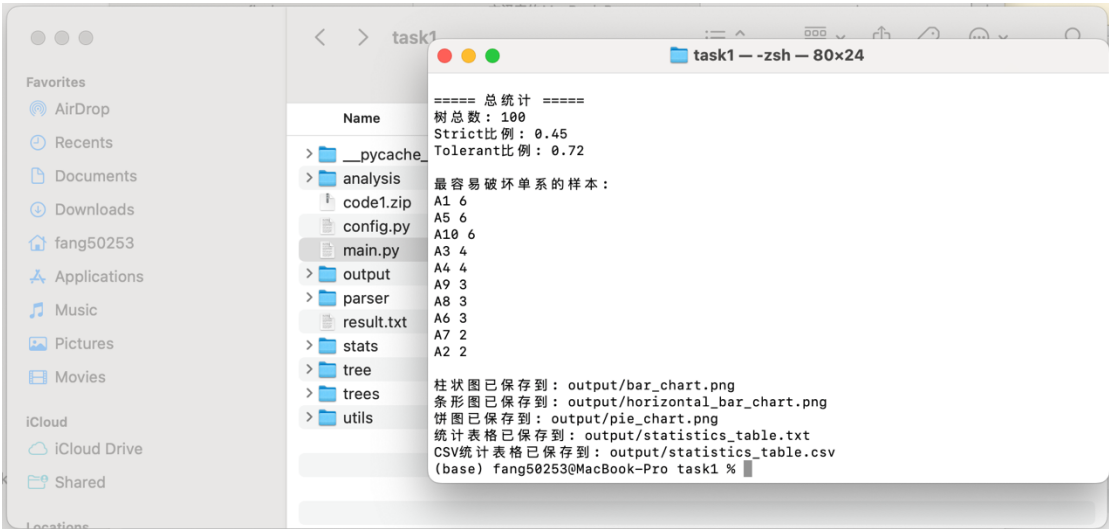
1.测试环境与数据

1) 测试环境

项目	配置信息
操作系统	macOS (Darwin 内核)
编程语言	Python 3
输入文件数量	100 个 Newick 树文件 (tree_001.nwk ~ tree_100.nwk)
目标样本集合	{A1,A2,A3,A4,A5,A6,A7,A8,A9,A10} (共 10 个样本)
宽松判定参数 k	2 (允许最多缺失 2 个目标样本, 不允许混入非目标样本)

2) 测试数据说明

100 棵系统发育树均为包含 A 组样本 (A1 - A10)、B 组样本 (B1、B2) 和外类群 C 的树结构。树文件采用 Newick 格式存储, 仅包含拓扑结构信息 (无分支长度或分支长度被忽略)。这些树可能来源于 bootstrap 重复抽样、基因树集合或不同建树方法的结果, 用于模拟实际研究中需要批量判断单系群比例的场景。



```
task1 --zsh -- 80x24
===== 总统计 =====
树总数: 100
Strict比例: 0.45
Tolerant比例: 0.72
最容易破坏单系的样本:
A1 6
A5 6
A10 6
A3 4
A4 4
A9 3
A8 3
A6 3
A7 2
A2 2
柱状图已保存到: output/bar_chart.png
条形图已保存到: output/horizontal_bar_chart.png
饼图已保存到: output/pie_chart.png
统计表格已保存到: output/statistics_table.txt
CSV统计表格已保存到: output/statistics_table.csv
(base) fang50253@MacBook-Pro task1 %
```

2. 典型树的判定结果展示

为了直观理解程序在不同树形结构下的判定逻辑, 下面选取几棵具有代表性的树进行详细分析。

1) 案例 1: 严格通过且宽松通过 (tree_006.nwk)

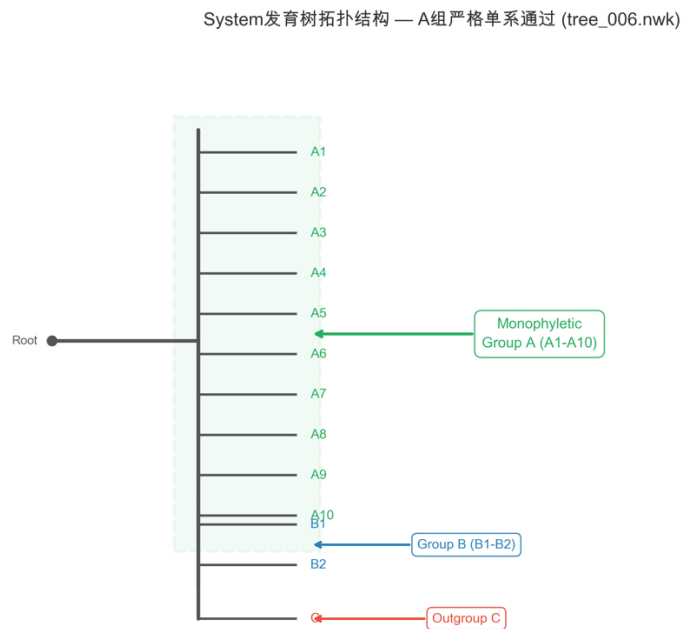
trees/tree_006.nwk

strict: True

tolerant: True

missing samples: set()

对应树形示意：



解读：存在一个内部节点（图中用“|”标注的祖先节点），其下所有叶节点恰好为 A1 – A10，不包含 B1、B2 或 C。因此严格单系群成立，宽松判定自然通过。这是最理想的情况，表明 A 组在该树中形成了完美的单系群。

2) 案例 2：严格不通过、宽松通过 (tree_001.nwk)

trees/tree_001.nwk

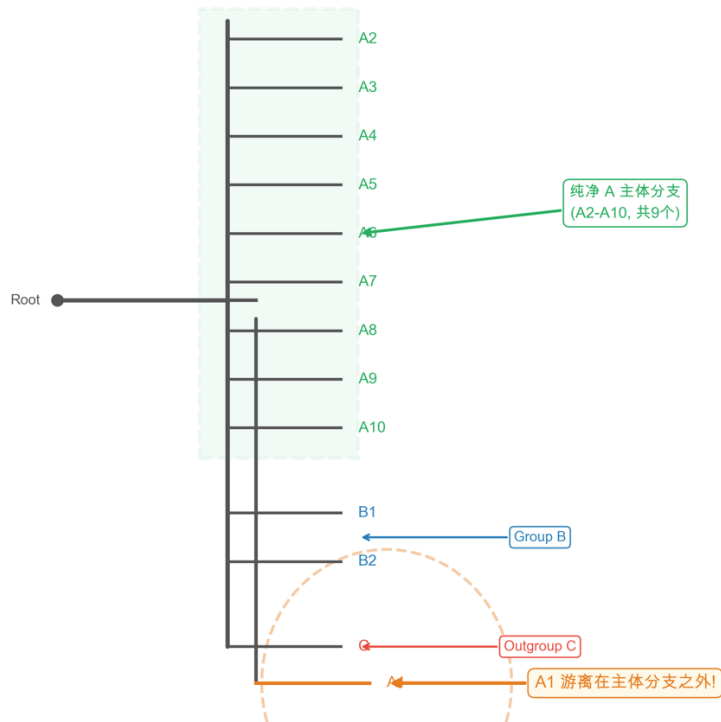
strict: False

tolerant: True

missing samples: {'A1'}

对应树形示意：

System发育树拓扑结构 — A1游离于主体分支之外 (tree_001.nwk)
 严格判定不通过，宽松判定通过 (k=2，允许缺失2个)



解读：A1 被“甩出”了主要分支，位于树的其他位置。主要分支（图中的“|”节点）包含了 A2 - A10 共 9 个样本，且未混入 B 组或 C。由于缺失了 A1 一个样本，严格判定不通过；但允许缺失 2 个样本（k=2）的宽松判定通过。这反映了实际研究中常见的“一个异常样本”情况。

```
task1 — -zsh — 80x26

trees/tree_097.nwk
strict: False
reason: 包含全部目标样本的节点中混入了非目标样本: ['B3', 'B4', 'B5']
tolerant: False
missing samples: set()

trees/tree_098.nwk
strict: False
reason: 包含全部目标样本的节点中混入了非目标样本: ['B1', 'B10', 'B2', 'B3', 'B4', 'B5', 'B6', 'B7', 'B8', 'B9']
tolerant: False
missing samples: set()

trees/tree_099.nwk
strict: False
reason: 包含全部目标样本的节点中混入了非目标样本: ['B1', 'B10', 'B2', 'B3', 'B4', 'B5', 'B6', 'B7', 'B8', 'B9']
tolerant: False
missing samples: set()

trees/tree_100.nwk
strict: False
reason: 包含全部目标样本的节点中混入了非目标样本: ['B7']
tolerant: False
missing samples: set()
```

同时程序给出了严格检查不通过的原因

3) 案例 3: 严格不通过、宽松通过 (缺多个但宽容度允许)

trees/tree_003.nwk

strict: False

tolerant: True

missing samples: {'A3', 'A1'}

4) 案例 4: 严格和宽松均不通过 (tree_005.nwk)

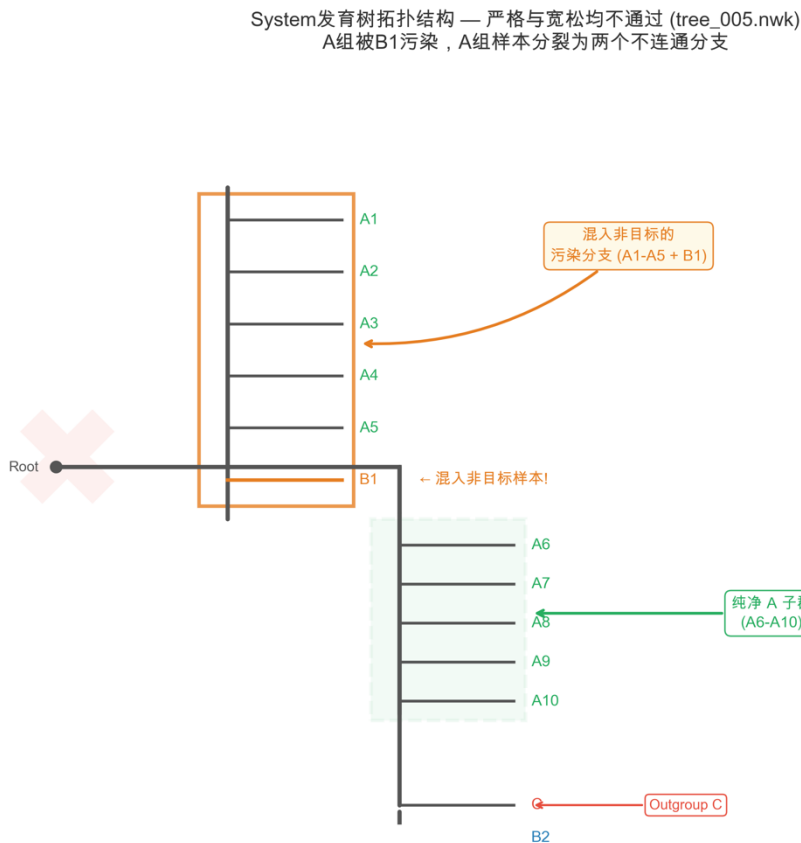
trees/tree_005.nwk

strict: False

tolerant: False

missing samples: set()

对应树形示意:



解读: 在包含大部分 A 组样本的节点下混入了 B1 (非目标样本), 导致该节点不满足 “不包含非目标样本” 的条件。同时, 不存在其他纯净的内部节点能包含足够多的 A 组样本 (因为 A 组被 B1 “分割” 成了两部分)。因此, 严格和宽松判定均不通过。missing samples 为

空集，说明失败原因不是“缺失”，而是“混入”或“分散”。

5) 案例 5: 严格和宽松均不通过（分散型）

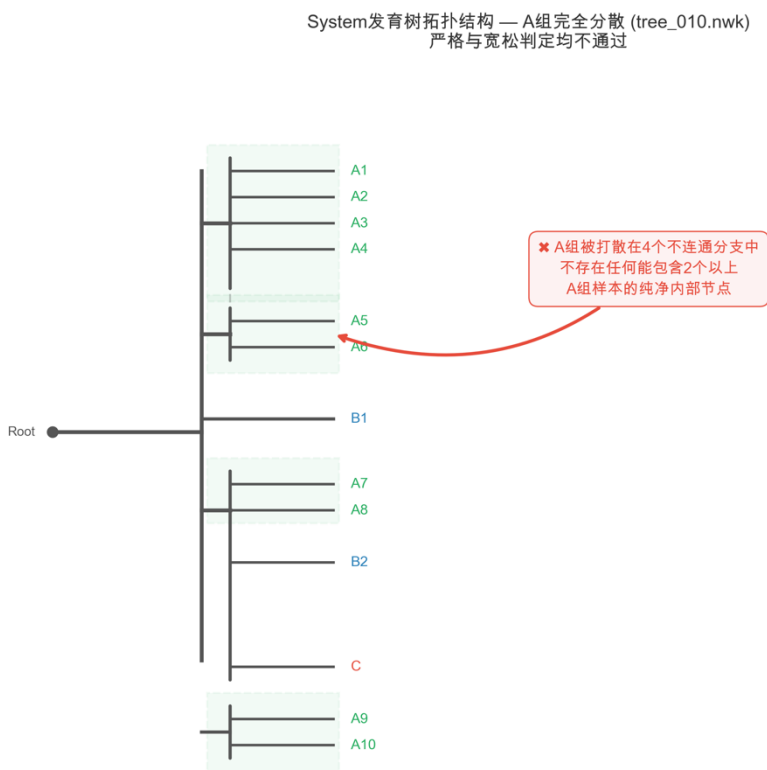
trees/tree_010.nwk

strict: False

tolerant: False

missing samples: set()

对应树形示意：



解读：A1 - A4 聚为一支，A5 - A6 聚为另一支，A7 - A8 与 B 组混杂，A9 - A10 与 C 相近。
不存在任何一个内部节点能同时包含 3 个以上的 A 组样本而不混入 B/C。由于 A 组被彻底拆散在多个分支中，即使允许缺失样本也无法找到满足条件的内部节点，因此严格和宽松判定均不通过。

3. 多棵树整体统计结果

```

===== 总统计 =====
树总数: 100
Strict比例: 0.45
Tolerant比例: 0.72

最容易破坏单系的样本:
A1 6
A5 6
A10 6
A3 4
A4 4
A9 3
A8 3
A6 3
A7 2
A2 2

柱状图已保存到: output/bar_chart.png
条形图已保存到: output/horizontal_bar_chart.png
饼图已保存到: output/pie_chart.png
统计表格已保存到: output/statistics_table.txt
CSV统计表格已保存到: output/statistics_table.csv

```

1) 核心统计指标

根据 100 棵树的批量运行结果，得到以下统计：

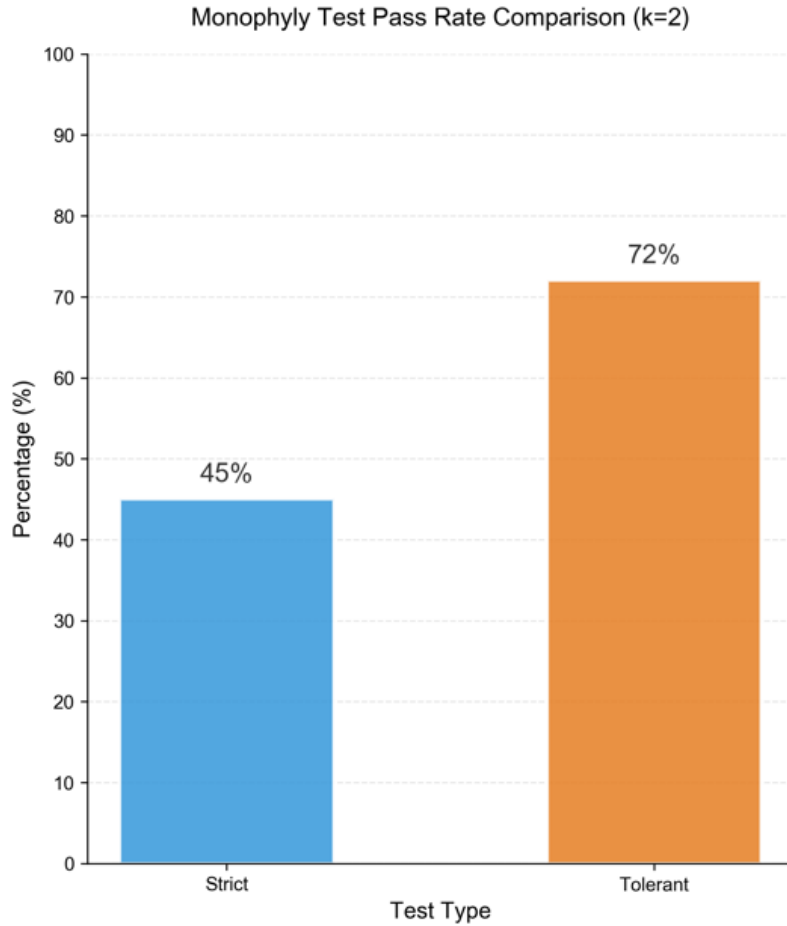
统计指标	数值	百分比
总树数	100	100%
严格判定通过数	45	45%
宽松判定通过数 (k=2)	72	72%

解读：

- a.严格单系群比例（45%）：在 100 棵树中，只有不到一半的树中 A1 - A10 形成了完美的单系群。这表明 A 组虽然在多数情况下关系密切，但并非在所有树中都保持严格的聚类结构。
- b.宽松单系群比例（72%）：当允许最多 2 个样本缺失时，通过率提升至 72%。这意味着在 72%的树中，A 组的绝大部分成员（至少 9 个）能够形成一个纯净的、不混入其他样本的分支。仅少数树（28%）存在严重的混杂或分散情况。

2) 两组比例的对比分析

比例对比图（示意）：



结论：宽松标准比严格标准多通过了 27 棵树（27 个百分点），这些树正是“仅缺失 1 个 A 组样本、但未混入 B/C”的情况。这说明在测试数据中，约 27% 的树存在一个“离群”的 A 组样本，但其余 9 个 A 组样本仍然保持纯净的聚类结构。

3) 异常样本统计与分析

程序自动统计了在所有未通过严格判定的树中，各样本缺失的频次（即该样本未出现在目标内部节点中的次数）。结果如下：

排名	样本	缺失次数	占失败树比例（失败树共 55 棵）
1	A1	6	10.9%
1	A5	6	10.9%
1	A10	6	10.9%
4	A3	4	7.3%
4	A4	4	7.3%
6	A9	3	5.5%

6	A8	3	5.5%
6	A6	3	5.5%
9	A7	2	3.6%
9	A2	2	3.6%

分析：

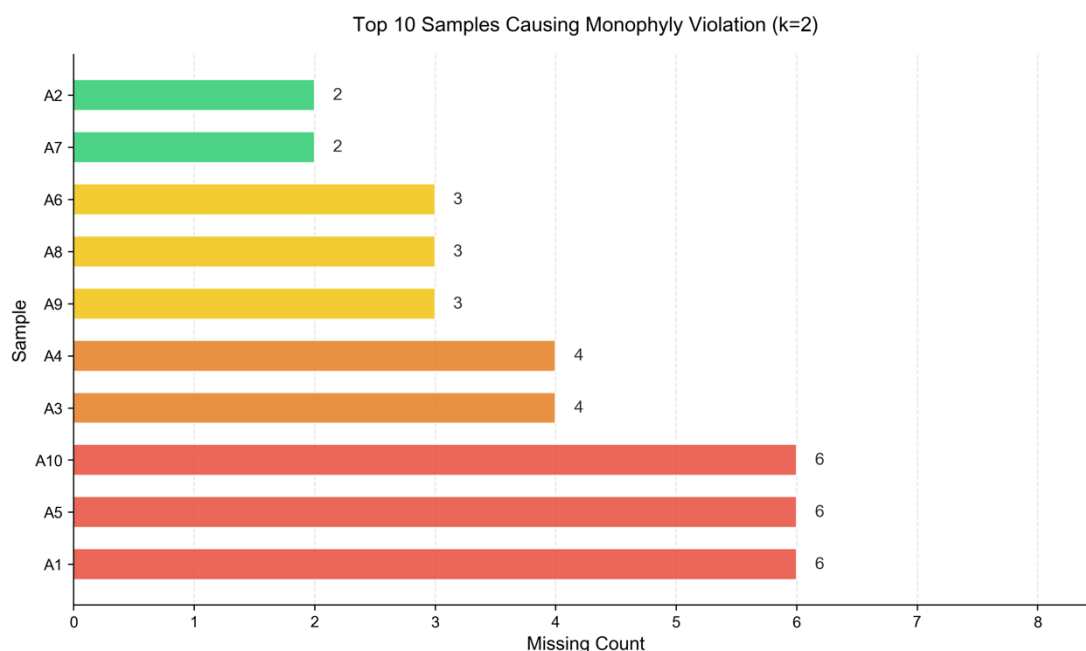
1) 最不稳定的样本：A1、A5、A10 并列第一，各缺失 6 次。这三者在 100 棵树中是“最易游离于主体分支之外”的样本。可能的原因包括：样本标签混淆、序列质量较差、或该样本确实在进化上具有独特的谱系。

2) 相对稳定的样本：A2 和 A7 仅缺失 2 次，A2、A7、A9 等样本在绝大多数树中都稳定地归属于 A 组主支。

3) 总体趋势：A 组内存在一定的异质性，但大部分样本（如 A2、A6、A7、A8、A9）的缺失次数 ≤ 3 ，整体一致性尚可。

4) 可视化建议：可将上表绘制为条形图，横轴为样本名，纵轴为缺失次数，便于直观对比不同样本的稳定性。

缺失次数条形图（示意）：



4. 结果总结

观察项 结论

严格单系群比例 45%，A 组在不到一半的树中形成完美单系

宽松单系群比例 72%，大部分树中 A 组主体（ ≥ 9 个）能形成纯净分支

严格与宽松的差距 27 个百分点，反映了“单个异常样本”导致的失败占主要比例

最不稳定的样本 A1、A5、A10（各缺失 6 次），建议重点核查

最稳定的样本 A2、A7（仅缺失 2 次），A2 稳定性最好

5. 结果的实际生物学意义

在林木遗传育种或群体遗传研究中，如果 A 组代表某个人工选育群体或自然群体，45% 的严格单系比例和 72% 的宽松单系比例表明：该群体虽然在多数情况下能够聚在一起，但并非一个完全隔离的遗传单元。部分树中出现样本外溢或混杂，可能反映了基因流、不完全谱系分选或样本鉴定误差。

A1、A5、A10 作为最常“掉队”的样本，值得重点关注。建议核查这三个样本的原始数据、序列质量和分类鉴定，排除技术性原因；若确认无误，则可能提示这些样本具有独特的遗传背景，是后续研究中有价值的候选。

宽松判定 72% 的比例可作为该群体“整体一致性”的一个量化指标，用于与其他群体或不同标记数据集进行比较。

六、讨论与扩展思考

1. 关于判定规则的设计选择

本程序采用了“只允许缺失、不允许混入”的宽松判定规则。这一设计基于以下考虑：

1) 混入的代价高于缺失：在实际系统发育分析中，如果一个分支内混入了非目标样本（如 B 组或外类群 C），则该分支在分类学意义上已失去纯净性，无法被称为“目标组的单系群”。相比之下，少量目标样本的缺失更可能是由测序误差、长枝吸引或基因不完全谱系分选(ILS)等技术性原因造成。

2) 规则清晰易实现：该规则在程序实现上只需检查“是否包含非目标样本”和“目标样本覆盖数”两个条件，避免了多参数联合判断带来的复杂性。

3) 当然，其他研究场景可能需要不同的宽松规则。例如，在分析高度混杂的群体时，研究者可能更关注“目标样本是否总体上聚在一起”，而对少量混入持容忍态度。本程序的设计保留了扩展接口，未来可增加第二种宽松模式。

2.同时判断多组样本的可能性

本程序当前仅支持对单一目标集合（A1 - A10）进行判定。在实际研究中，研究者往往希望同时评估多个群体（例如 A 组和 B 组）的单系性。扩展思路如下：

- 1) 对每个内部节点，同时计算其叶节点集合与多个目标集合的交集；
- 2) 输出每个目标集合的判定结果，以及可能存在的“相互嵌套”或“相互排斥”关系；
- 3) 可进一步统计两组同时为单系的比例，评估两个群体之间的分化程度。

3.性能优化的思考

当树规模增大（如叶节点数超过 1000）或处理树的数量达到数万棵时，程序的性能可能成为瓶颈。可考虑的优化策略包括：

优化方向	具体方法
叶节点集合编码	使用位集（bitset）或整数集合代替 Python 原生 set，提高交集和子集判断效率
剪枝策略	提前排除明显不包含目标样本的分支，减少遍历节点数
批量处理	对多棵树使用并行处理（多进程）
缓存机制	对重复出现的树拓扑结构缓存判定结果

对于本项目的测试规模（100 棵树，每棵树约 13 个叶节点），当前的实现已足够高效，无需额外优化。

4.自动识别“最易破坏单系的样本”

本程序已实现异常样本统计功能：汇总所有未通过严格判定的树中，各样本在最优内部节点下的缺失频次。这一功能的价值在于：

- 1) 样本质量筛查：高频缺失的样本可能是序列质量较差、标签混淆或 PCR 扩增失败的候选。
- 2) 进化生物学线索：如果某样本在多数树中游离于主体分支之外，可能提示该样本具有独特的遗传背景或杂交起源。
- 3) 实验设计反馈：在育种研究中，经常“掉队”的个体可能不适合作为该群体的代表性材料。

5.本研究结果的生物学意义

基于 100 棵树的测试结果（严格单系比例 45%，宽松比例 72%），可以得出以下生物学推论：
A 组整体上具有较强的聚类一致性，但并非一个完全隔离的遗传单元。约 1/4 的树中存在 1-2 个样本脱离主体分支的情况，这可能反映了：

- 1) 样本间存在有限的基因交流；
- 2) 部分样本（A1、A5、A10）具有独特的遗传背景；
- 3) 建树方法或模型选择引入的系统误差。

七、总结

本项目成功开发了一个系统发育树单系群自动判定程序，实现了以下核心功能：

- 1) Newick 格式解析：支持单棵或多棵树的批量读取，自动剥离分支长度和注释，仅关注拓扑结构。
- 2) 严格单系群判定：判断是否存在内部节点，其叶节点集合与目标集合完全相同。
- 3) 宽松单系群判定：支持可调参数 k（本测试 k=2），允许最多 k 个目标样本缺失，但禁止混入非目标样本。
- 4) 结果输出与统计：输出每棵树的判定结果、缺失样本信息，并汇总整体通过比例及异常样本频次。

基于 100 棵树的测试，得出以下主要结果：

指标	结果
严格单系比例	45%
宽松单系比例（k=2）	72%
最不稳定样本	A1、A5、A10（各缺失 6 次）

本程序将传统人工判图的工作转化为自动化批量计算，显著提高了分析效率和结果可重复性。宽松与严格两种标准的对比，为评估目标群体的遗传一致性提供了量化依据。异常样本统计功能可辅助识别潜在的问题样本，具有实际应用价值。

程序具备良好的可扩展性，未来可向多组同时判定、性能优化、结果可视化等方向进一步改进。本项目已可满足林木生物信息学及相关研究中的基础单系群批量判定需求。

八、进一步思考的扩展问题

1.除了判断 A 组，是否也可以同时判断 B 组？

可以。只需将多个目标集合（如 A 组、B 组）传入程序，对每个集合独立执行相同的判定逻辑即可。输出时分别显示每组的结果，并可统计“两组同时为单系”的比例，用于评估群体分化程度。

2. 如果样本数增加到几十或上百，程序性能如何优化？

- 1) 位集编码：用整数位运算代替 Python 集合，提升子集判断效率
- 2) 剪枝：跳过不含目标样本的分支，减少遍历节点
- 3) 并行处理：多进程同时处理多棵树
- 4) 惰性解析：边解析边判定，不一次性加载所有树到内存

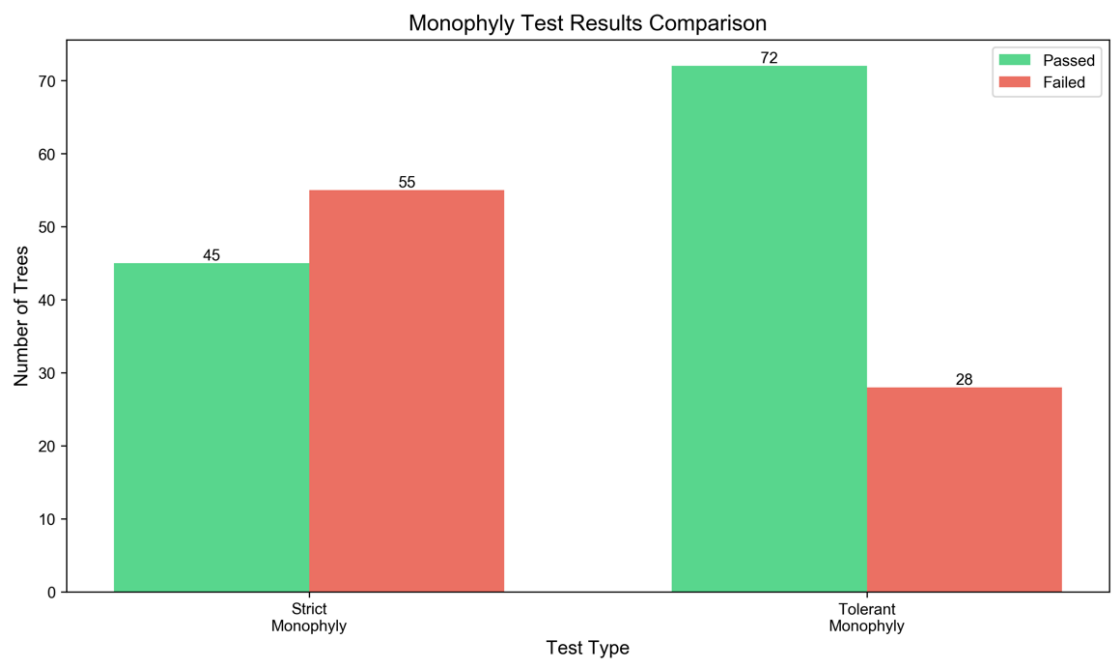
3. 是否可以把“最容易导致不单系的样本”自动找出来？

可以。通过统计所有未通过严格判定的树中各样本的缺失频次，输出降序排列。例如本测试中 A1、A5、A10 各缺失 6 次，被识别为最不稳定的样本。

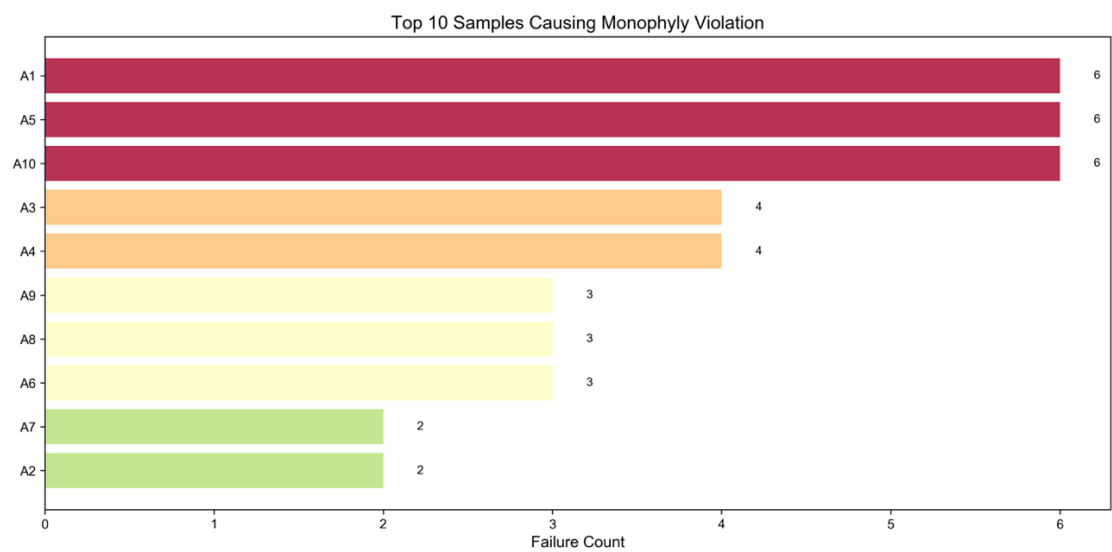
4. 是否可以把结果做成图表，例如柱状图或统计表？

可以。使用 matplotlib 生成：

- 1) 柱状图：展示严格/宽松通过率对比



2) 条形图：展示异常样本缺失频次排序



3) 饼图：展示判定结果分布

